

Many-to-many Object Relationships

 <https://github.com/learn-co-curriculum/p3-oo-many-to-many>  <https://github.com/learn-co-curriculum/p3-oo-many-to-many/issues/new>

Learning Goals

- Describe many-to-many relationships.
- Explain why many-to-many relationships are important.

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.

Introduction

A many-to-many relationship refers to a relationship where multiple instances of one class (**User** s) are associated with multiple instances of another class (**Website** s), and vice versa. In other words, each object of **User** s can be related to multiple objects of Websites, and each object of **Website** s can be related to multiple objects of **User** s.

When working with many-to-many relationships, it is important to consider whether or not to use an intermediary class. An intermediary class is a class that sits between two other classes in a many-to-many relationship and provides additional information about the relationship. It can help us populate and modify both sides of the relationship.

Many-to-many without intermediary class

An approach to many-to-many relationships without using an intermediary class is to use a list to store references to the related objects.

Lets look at an example where we use a `Parent` and `Child` class to demonstrate a many-to-many relationship. A child can have multiple parents and a parent can have multiple children.

```
class Parent:

    all = []

    def __init__(self, name, children=None):
        self.name = name
        self._children = []
        if children:
            for child in children:
                self.add_child(child)
        Parent.all.append(self)

    def children(self):
        return self._children

    def add_child(self, child):
        if isinstance(child, Child):
            self._children.append(child)
        else:
            raise ValueError("Child must be an instance of the Child class.")

class Child:

    def __init__(self, name):
        self.name = name
```

```
def parents(self):
    return [parent for parent in Parent.all if self in parent.children()]

def add_parent(self, parent):
    if isinstance(parent, Parent):
        parent.add_child(self)
    else:
        raise ValueError("Parent must be an instance of the Parent class.")
```

Note that while the two classes are related to each other in the same way, we took different approaches to accessing related objects in each class. This is to maintain our single source of truth (SSOT): the relationship is stored in the `Parent` class, but both classes are designed such that their related objects are accessed in the same way.

```
parent1 = Parent('Nick')
parent2 = Parent('Megan')
child1 = Child('Steve')
child2 = Child('Liz')

parent1.add_child(child1)
parent2.add_child(child1)
child2.add_parent(parent1)
child2.add_parent(parent2)
[ print(c.name) for c in parent1.children()]
# => Steve
# => Liz
[ print(p.name) for p in child1.parents()]
# => Nick
# => Megan
```

This approach can be simpler than using an intermediary class, but it can also be less flexible if additional information needs to be stored about the relationship.

Many-to-many with an Intermediary Class

In this example, we'll create a many-to-many relationship between two classes `Student` and `Course`, using a third intermediary class called `Enrollment`.

```
from datetime import datetime
```

```
class Student:
```

```
    all = []
```

```
    def __init__(self, name):
        self.name = name
        Student.all.append(self)
```

```
    def enroll_in_course(self, course):
        Enrollment(self, course)
```

```
    def enrollments(self):
        return [enrollment for enrollment in Enrollment.all if enrollment.student == self]
```

```
    def courses(self):
        return [enrollment.course for enrollment in self.enrollments()]
```

```
class Course:
```

```
    all = []
```

```
    def __init__(self, title):
        self.title = title
        Course.all.append(self)
```

```
def enrollments(self):
    return [enrollment for enrollment in Enrollment.all if enrollment.course == self]

def students(self):
    return [enrollment.student for enrollment in self.enrollments()]

def enroll_student(self, student):
    Enrollment(student, self)

class Enrollment:

    all = []

    def __init__(self, student, course):
        self.student = student
        self.course = course
        self.enrollment_date = datetime.now()
        Enrollment.all.append(self)
```

Using an intermediary class, you can now store the date when a student enrolls in a course as part of the Enrollment object, which is specific to the relationship between the student and the course. The tradeoff is that it adds complexity because you need to create and manage additional objects. The advantage is that it makes the code more scalable because if we need to add more attributes or features related to the enrollment process we can easily do so with the `Enrollment` class.

```
student = Student('Steve')
course = Course('Math 31')

student.enroll_in_course(course)
print(student.enrollments()[0].enrollment_date)
# => 2023-05-02 08:39:57.570467
```

```
print(course.enrollments()[0].enrollment_date)
# => 2023-05-02 08:39:57.570467
print(student.enrollments()[0].course.title)
# => Math 31
```

Conclusion

In conclusion, many-to-many relationships are an important aspect of object-oriented programming that enable us to model complex relationships between entities. These relationships can be implemented with or without an intermediary class. The decision to use an intermediary class depends on the specific requirements of the system and the information that needs to be stored about the relationship.

When using an intermediary class, it is possible to store additional information about the relationship that is not directly related to either of the entities involved. This information can include data such as timestamps, ratings, or any other attributes that are relevant to the relationship itself. By utilizing an intermediary class, we can model these relationships in a more robust and flexible way.

However, using an intermediary class can also introduce additional complexity and overhead to the system. This can make the system more difficult to maintain, especially if there are many entities and relationships involved. In some cases, it may be more efficient to implement the many-to-many relationship without an intermediary class, especially if the relationship is simple and straightforward (such as with parents and children).

In summary, many-to-many relationships are a critical aspect of object-oriented programming that enable us to model complex relationships between entities. The use of an intermediary class can provide additional flexibility and robustness, but also introduces additional complexity and overhead. Careful consideration of these trade-offs is essential when deciding how to implement many-to-many relationships in our programs.

Resources

- [Python classes](https://docs.python.org/3/tutorial/classes.html)  [_\(https://docs.python.org/3/tutorial/classes.html\)](https://docs.python.org/3/tutorial/classes.html)